

Table of Contents

Guía de Preparación para Entrevista Técnica.....	1
--	---

Guía de Preparación para Entrevista Técnica

Java · Spring Boot · Microservicios · APIs REST · Bases de Datos · Kafka · Docker/Kubernetes

1. Java Core

¿Cuál es la diferencia entre HashMap y ConcurrentHashMap?

- **HashMap:** no es thread-safe. Si varios hilos lo modifican concurrentemente puede producir `ConcurrentModificationException` o corromper su estructura interna (bucles infinitos en versiones antiguas).
- **ConcurrentHashMap:** diseñado para concurrencia. En Java 8+ usa segmentación a nivel de “bucket” con bloqueos finos (CAS + `synchronized` por nodo) en lugar de bloquear todo el mapa. Permite lecturas concurrentes sin bloqueo y escrituras concurrentes en distintos buckets.
- **Null keys/values:** HashMap permite una key null y varios values null; ConcurrentHashMap no permite null en ninguno (para evitar ambigüedad entre “no existe” y “es null” en un entorno concurrente).
- **Uso:** HashMap para código single-thread; ConcurrentHashMap cuando el mapa se comparte entre hilos (cachés, contadores compartidos, etc.).

¿Qué diferencias existen entre una interfaz y una clase abstracta?

Aspecto	Interfaz	Clase abstracta
Herencia múltiple	Una clase puede implementar varias	Solo puede extender una
Estado (campos)	Solo constantes (<code>public static final</code>)	Puede tener atributos de instancia
Constructores	No tiene	Sí puede tener
Métodos	Abstractos, <code>default</code> y <code>static</code> (Java 8+)	Abstractos y concretos
Propósito	Definir un contrato (“qué puede hacer”)	Definir una jerarquía compartida (“qué es” + comportamiento común)

Regla práctica: si necesitas compartir código/estado entre clases relacionadas, usa clase abstracta; si solo necesitas definir un contrato que clases no relacionadas puedan cumplir, usa interfaz.

¿Qué es el Garbage Collector y cómo funciona?

Es el mecanismo de la JVM que libera automáticamente memoria de objetos que ya no son alcanzables (no tienen referencias vivas), evitando fugas de memoria y gestión manual como en C/C++.

Funcionamiento general (generacional): 1. **Young Generation** (Eden + Survivor spaces): la mayoría de objetos mueren jóvenes. Minor GC frecuente y rápido. 2. **Old Generation (Tenured)**: objetos que sobreviven varios ciclos se promueven aquí. Major/Full GC, más costoso. 3. Algoritmo general: **Mark** (marcar objetos alcanzables desde GC Roots) → **Sweep** (liberar no alcanzables) → **Compact** (opcional, defragmenta memoria).

Recolectores modernos: **G1GC** (default desde Java 9, balancea throughput y pausas), **ZGC** y **Shenandoah** (pausas sub-milisegundo, útiles para heaps enormes).

¿Cuál es la diferencia entre == y .equals()?

- **==**: compara referencias (¿es el mismo objeto en memoria?) para tipos objeto; compara valores para tipos primitivos.
- **.equals()**: compara el contenido/valor lógico según cómo se sobrescriba en la clase. Por defecto (`Object.equals`) se comporta igual que **==**, pero clases como `String`, `Integer`, o clases de dominio suelen sobrescribirlo para comparar valores.
- Trampa clásica: `new String("a") == new String("a")` → `false`, pero `.equals()` → `true`.
- Regla: siempre que sobrescribas `equals()`, sobrescribe también `hashCode()` (contrato de `Object`), para que la clase funcione correctamente en `HashMap/HashSet`.

¿Qué son las colecciones en Java y cuándo usarías List, Set y Map?

El Java Collections Framework provee estructuras de datos estándar con interfaces comunes.

- **List** (`ArrayList`, `LinkedList`): colección ordenada, permite duplicados, acceso por índice. Úsala cuando el orden de inserción importa o necesitas acceso posicional (`ArrayList` para lecturas frecuentes, `LinkedList` para inserciones/eliminaciones frecuentes en los extremos).
- **Set** (`HashSet`, `LinkedHashSet`, `TreeSet`): no permite duplicados. Úsalo cuando necesitas garantizar unicidad (`HashSet` para rendimiento, `LinkedHashSet` si necesitas preservar orden de inserción, `TreeSet` si necesitas orden natural/ordenado).
- **Map** (`HashMap`, `LinkedHashMap`, `TreeMap`): pares clave-valor, claves únicas. Úsalo para lookups rápidos por clave, cachés, o representar relaciones clave→valor.

¿Qué es la programación funcional en Java?

Paradigma introducido fuertemente desde Java 8 que trata funciones como ciudadanos de primera clase. En Java se implementa mediante: - **Interfaces funcionales** (`Function`, `Predicate`, `Supplier`, `Consumer`, etc.), interfaces con un solo método abstracto. - **Lambdas**

y **method references** para pasar comportamiento como argumento. - **Streams** para procesar colecciones de forma declarativa. - Fomenta funciones puras, inmutabilidad y composición sobre mutación de estado, aunque Java sigue siendo un lenguaje híbrido (OOP + funcional), no puramente funcional.

¿Qué ventajas aportan las expresiones Lambda?

- **Código más conciso:** eliminan boilerplate de clases anónimas.
- **Legibilidad:** expresan intención ("qué hacer") en lugar de mecánica ("cómo iterar").
- **Facilitan Streams y APIs funcionales** (`Comparator.comparing(...)`, `map`, `filter`, etc.).
- **Comportamiento como parámetro:** permiten pasar lógica a métodos fácilmente (callbacks, estrategias).
- Se compilan de forma eficiente (usan `invokedynamic`, no generan una clase por cada lambda como ocurría con clases anónimas).

¿Qué son los Streams y qué beneficios tienen?

Los Streams (`java.util.stream`) son una abstracción para procesar secuencias de datos de forma **declarativa** y potencialmente **paralela**, sin modificar la fuente original.

Beneficios: - Encadenamiento fluido de operaciones (`filter`, `map`, `sorted`, `collect`, `reduce`). - Separan **qué** se quiere lograr de **cómo** se itera. - Operaciones **lazy** (intermedias no se ejecutan hasta que hay una operación terminal), lo que permite optimizaciones. - Soporte para paralelismo casi gratis con `parallelStream()`. - Código más legible que loops anidados con condicionales.

Ejemplo:

```
List<String> nombres = personas.stream()
    .filter(p -> p.getEdad() > 18)
    .map(Persona::getNombre)
    .sorted()
    .collect(Collectors.toList());
```

Explica el concepto de Inmutabilidad.

Un objeto inmutable es aquel cuyo estado no puede cambiar una vez creado (ej. `String`, `Integer`, clases record sin setters).

Cómo lograrla: - Declarar la clase `final` (evita subclasses que rompan invariantes). - Todos los campos `private final`. - No exponer setters; inicializar todo en el constructor. - Si un campo es una colección o referencia mutable, devolver copias defensivas en los getters.

Ventajas: - **Thread-safety** automática (no hay estado mutable compartido que sincronizar). - Más fácil de razonar (no hay efectos secundarios inesperados). - Seguros como claves de `HashMap`/elementos de `HashSet` (su `hashCode` no cambia). - Java 16+ introdujo record para crear clases inmutables de forma muy concisa.

¿Qué cambios importantes conoces de Java 8, 17 y 21?

Java 8 (2014) — el salto más grande: - Lambdas y expresiones funcionales. - Streams API. - `Optional<T>` para evitar `NullPointerException`. - Nueva API de fecha/hora (`java.time: LocalDate, LocalDateTime, etc.`). - Métodos `default` y `static` en interfaces.

Java 17 (2021, LTS): - `record` (clases inmutables concisas para DTOs/value objects). - `sealed classes` (jerarquías de herencia restringidas y controladas). - Pattern matching para `instanceof`. - Text blocks (`"""..."""`) para strings multilínea. - Switch expressions mejoradas.

Java 21 (2023, LTS): - **Virtual Threads** (Project Loom): hilos ligeros gestionados por la JVM, permiten concurrencia masiva sin el costo de hilos del SO — clave para I/O intensivo en microservicios. - Pattern matching para `switch` (con deconstrucción de records). - Sequenced Collections (nuevo orden garantizado de acceso a primer/último elemento). - Structured Concurrency (preview) para manejar tareas concurrentes como una unidad.

2. Spring Boot

¿Qué es la Inversión de Control (IoC)?

Principio de diseño donde el control de creación y gestión del ciclo de vida de los objetos (dependencias) se delega a un contenedor externo (el **Spring IoC Container**), en lugar de que el propio objeto las cree con `new`. Esto invierte el flujo tradicional: en lugar de que la clase controle sus dependencias, el framework se las “inyecta” (**Dependency Injection**, la implementación más común de IoC en Spring). Beneficios: bajo acoplamiento, mayor testabilidad (se pueden inyectar mocks), y configuración centralizada.

¿Cuál es la diferencia entre `@Component`, `@Service` y `@Repository`?

Los tres son especializaciones de `@Component` y hacen que Spring detecte la clase como un bean gestionado (component scanning). La diferencia es semántica y de comportamiento adicional: - **@Component**: genérico, para cualquier bean gestionado por Spring. - **@Service**: marca la capa de lógica de negocio. Semánticamente indica “esto es un servicio”. - **@Repository**: marca la capa de acceso a datos. Además, Spring aplica **traducción automática de excepciones** (convierte excepciones específicas de JDBC/JPA en `DataAccessException`, una jerarquía unificada de Spring).

En la práctica funcional son intercambiables, pero usar la anotación correcta mejora la legibilidad y habilita comportamientos extra (como en `@Repository`).

¿Qué es Spring Boot y qué ventajas ofrece?

Spring Boot es un framework construido sobre Spring que simplifica drásticamente la configuración y el arranque de aplicaciones Spring.

Ventajas principales: - **Auto-configuración**: configura beans automáticamente según las dependencias en el classpath (convención sobre configuración). - **Starters**: dependencias

empaquetadas (spring-boot-starter-web, -data-jpa, -security) que evitan gestionar versiones manualmente. - **Servidor embebido** (Tomcat/Jetty/Undertow): no requiere desplegar un WAR en un servidor externo. - **Spring Actuator**: métricas, health checks y monitoreo listos para producción. - **Menos boilerplate XML**: configuración vía anotaciones o application.yml.

¿Cómo manejarías la configuración entre distintos ambientes?

- **Perfiles de Spring** (@Profile, application-dev.yml, application-prod.yml) activados vía spring.profiles.active.
- **Variables de entorno** para secretos y valores sensibles (nunca hardcodeados ni en el repo).
- **Externalización de configuración**: application.yml con spring.config.import, o un servidor de configuración centralizado (Spring Cloud Config) en arquitecturas de microservicios.
- Jerarquía de precedencia de Spring: argumentos de línea de comandos > variables de entorno > application-{profile}.yml > application.yml (por defecto), lo que permite sobrescribir valores por ambiente sin cambiar código.

¿Qué es Spring Security?

Framework de seguridad para aplicaciones Spring que provee autenticación (quién eres) y autorización (qué puedes hacer), protección contra ataques comunes (CSRF, session fixation, clickjacking), e integración con múltiples mecanismos: formularios, Basic Auth, OAuth2/OIDC, JWT, LDAP, etc. Se configura mediante una cadena de filtros (SecurityFilterChain) que intercepta cada request antes de llegar al controlador.

¿Cómo implementarías autenticación JWT?

Flujo típico: 1. Usuario envía credenciales a /login. 2. El servidor valida contra la base de datos/UserDetailsService, y si son correctas, genera un **JWT firmado** (con una librería como jjwt) que incluye claims (usuario, roles, expiración) en el payload. 3. El cliente guarda el token y lo envía en cada request en el header Authorization: Bearer <token>. 4. Un **filtro personalizado** (OncePerRequestFilter) intercepta cada petición, valida la firma y expiración del token, y si es válido, construye el Authentication y lo coloca en el SecurityContext. 5. Spring Security autoriza el acceso a los endpoints según los roles/authorities del token.

Puntos clave: usar refresh tokens para renovar sesiones sin re-loguear, mantener el secreto de firma fuera del código (variables de entorno/vault), y definir expiración corta para el access token.

¿Qué es Spring Data JPA?

Módulo de Spring que simplifica el acceso a datos relacionales sobre JPA/Hibernate. Permite definir repositorios como interfaces (extends JpaRepository<Entity, ID>) sin implementar el CRUD manualmente. Soporta **query methods derivados del nombre**

(findByEmailAndActiveTrue), @Query con JPQL o SQL nativo, paginación (Pageable) y ordenamiento listos para usar.

¿Cuál es la diferencia entre CrudRepository y JpaRepository?

- **CrudRepository**: interfaz base con operaciones CRUD básicas (save, findById, findAll, delete, etc.).
- **JpaRepository**: extiende PagingAndSortingRepository (que a su vez extiende CrudRepository), y agrega funcionalidades específicas de JPA: paginación, ordenamiento, flush(), operaciones batch (saveAll, deleteAllInBatch), y devuelve resultados en List en lugar de Iterable.
- En la práctica, para proyectos con JPA/Hibernate casi siempre se usa JpaRepository directamente, salvo que se quiera una abstracción de persistencia más genérica (por ejemplo, para soportar MongoDB u otro store también).

¿Qué son los perfiles (@Profile)?

Mecanismo para activar/desactivar beans o configuraciones según el ambiente de ejecución (dev, test, staging, prod). Se anota un @Bean o @Component con @Profile("dev") y solo se registrará si ese perfil está activo (spring.profiles.active=dev). Permite, por ejemplo, usar una base de datos H2 en memoria en desarrollo y PostgreSQL real en producción, sin cambiar código, solo configuración.

¿Cómo implementarías un manejo global de excepciones?

Usando @RestControllerAdvice junto con métodos @ExceptionHandler para centralizar el manejo de errores en toda la API:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse>
handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(ex.getMessage(),
HttpStatus.NOT_FOUND.value()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse>
handleValidation(MethodArgumentNotValidException ex) {
        // extraer errores de @Valid y devolver 400
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(new ErrorResponse("Error interno", 500));
    }
}
```

```
}  
}
```

Ventajas: evita repetir try-catch en cada controlador, respuestas de error consistentes (formato estándar tipo ErrorResponse), y separación clara entre lógica de negocio y manejo de errores.

3. Microservicios

¿Qué ventajas tienen los microservicios frente a un monolito?

- **Despliegue independiente:** cada servicio se despliega sin afectar a los demás.
- **Escalabilidad selectiva:** escalas solo el servicio que lo necesita, no toda la aplicación.
- **Aislamiento de fallos:** un fallo en un servicio no necesariamente tumba todo el sistema (si se maneja bien).
- **Libertad tecnológica:** cada servicio puede usar el stack más adecuado para su dominio.
- **Equipos autónomos:** equipos pequeños pueden poseer un servicio de punta a punta.

Trade-offs (vale la pena mencionarlos): mayor complejidad operativa, necesidad de observabilidad robusta, consistencia eventual en vez de transacciones ACID simples, y overhead de comunicación en red.

¿Qué patrones conoces para comunicación entre microservicios?

- **API Gateway:** punto de entrada único que enruta a los servicios internos.
- **Request/Response síncrono** (REST, gRPC).
- **Event-driven / Pub-Sub** (Kafka, RabbitMQ) para desacoplar productores y consumidores.
- **Saga pattern:** para transacciones distribuidas (coreografía u orquestación).
- **Circuit Breaker / Bulkhead:** para tolerancia a fallos.
- **BFF (Backend for Frontend):** un backend específico por tipo de cliente (web, móvil).
- **Service Mesh** (Istio, Linkerd): capa de infraestructura para manejar comunicación, seguridad y observabilidad entre servicios de forma transparente.

¿Cuál es la diferencia entre comunicación síncrona y asíncrona?

- **Síncrona** (REST, gRPC): el cliente espera la respuesta antes de continuar. Más simple de razonar, pero acopla la disponibilidad de un servicio a la del otro (si el servicio destino cae, la llamada falla).
- **Asíncrona** (mensajería vía Kafka/RabbitMQ): el productor envía un mensaje y continúa sin esperar respuesta inmediata; el consumidor procesa cuando puede. Mejora resiliencia y desacopla servicios, a cambio de mayor complejidad (consistencia eventual, necesidad de idempotencia, trazabilidad más difícil).

¿Qué es un API Gateway?

Componente que actúa como punto de entrada único para los clientes, ocultando la complejidad interna de los microservicios. Responsabilidades típicas: enrutamiento de requests al servicio correcto, autenticación/autorización centralizada, rate limiting, balanceo de carga, agregación de respuestas de varios servicios, y transformación de requests/responses. Ejemplos: Spring Cloud Gateway, Kong, AWS API Gateway, Nginx.

¿Qué es Service Discovery?

Mecanismo que permite a los servicios encontrarse dinámicamente entre sí sin hardcodear IPs/hostnames, algo crítico en entornos donde las instancias escalan y cambian de dirección constantemente (contenedores, Kubernetes).

- **Discovery del lado del cliente:** el servicio consulta un registro (ej. Eureka) y elige la instancia.
- **Discovery del lado del servidor:** un load balancer/proxy (ej. Kubernetes Service, Consul) hace el enrutamiento de forma transparente.

Herramientas: Netflix Eureka, Consul, o el DNS interno de Kubernetes.

¿Cómo manejarías transacciones distribuidas?

Evitando transacciones ACID clásicas entre servicios (2PC) por su acoplamiento y baja disponibilidad, y usando en su lugar el patrón **Saga**: - **Coreografía**: cada servicio publica eventos y reacciona a eventos de otros servicios (sin orquestador central); más desacoplado pero más difícil de rastrear el flujo completo. - **Orquestación**: un servicio orquestador coordina explícitamente los pasos y las compensaciones (rollbacks lógicos) si algo falla.

Complementarios: **outbox pattern** (para garantizar que un cambio de estado y la publicación de su evento sean atómicos) e **idempotencia** en cada paso para manejar reintentos con seguridad.

¿Qué es Circuit Breaker?

Patrón de resiliencia que previene que un servicio siga llamando a una dependencia que está fallando repetidamente, evitando efecto cascada. Funciona como un interruptor con tres estados: - **Closed**: las llamadas pasan normalmente; se cuentan fallos. - **Open**: tras superar un umbral de fallos, se “abre” y las llamadas fallan rápido (fail-fast) sin siquiera intentar la llamada real, dando tiempo al servicio afectado a recuperarse. - **Half-Open**: tras un tiempo, deja pasar algunas llamadas de prueba; si tienen éxito, vuelve a Closed, si no, vuelve a Open.

¿Qué es Resilience4j?

Librería ligera de tolerancia a fallos para Java/Spring Boot (sucesora espiritual de Netflix Hystrix, que quedó deprecado). Provee, de forma modular y componible: - **Circuit Breaker**

- **Retry** (reintentos con backoff) - **Rate Limiter** - **Bulkhead** (aislamiento de recursos, limitando llamadas concurrentes) - **Time Limiter** (timeouts)

Se integra fácilmente con Spring Boot mediante spring-boot-starter y anotaciones como @CircuitBreaker, @Retry.

¿Cómo implementarías observabilidad en microservicios?

Los tres pilares: 1. **Logging estructurado**: logs en formato JSON con correlationId/traceId para poder rastrear una request a través de varios servicios (esto conecta directamente con tu experiencia implementando logging HTTP hacia HRLOG en hogar-hr). 2. **Métricas**: exponer métricas vía Spring Boot **Actuator** + **Micrometer**, y visualizarlas en Prometheus/Grafana. 3. **Tracing distribuido**: propagar un trace ID entre servicios usando algo como **OpenTelemetry**, **Zipkin** o **Jaeger**, para ver el recorrido completo de una petición y dónde se generan cuellos de botella.

Adicional: centralizar logs con un stack tipo ELK (Elasticsearch, Logstash, Kibana) o Grafana Loki.

¿Qué métricas monitorearías en producción?

- **Latencia** (p50, p95, p99) por endpoint.
- **Tasa de errores** (4xx, 5xx).
- **Throughput** (requests por segundo).
- **Uso de recursos**: CPU, memoria, garbage collection (pausas de GC son clave en JVM).
- **Saturación de conexiones**: pool de base de datos, threads activos.
- **Salud de dependencias externas** (colas, otros servicios).
- **Métricas de negocio** específicas cuando aplique (ej. órdenes procesadas por minuto).

4. APIs REST

Diferencia entre PUT y PATCH.

- **PUT**: reemplaza el recurso completo. El cliente envía la representación completa; cualquier campo no incluido se interpreta como eliminado/null.
- **PATCH**: aplica una modificación **parcial** al recurso, enviando solo los campos que cambian.
- Ambos deberían ser **idempotentes**: aplicar la misma petición PUT o PATCH varias veces debería producir el mismo resultado final.

¿Qué códigos HTTP utilizas más frecuentemente?

- **200 OK**: éxito genérico (GET, PUT/PATCH exitoso).
- **201 Created**: recurso creado (POST exitoso), normalmente con header Location.
- **204 No Content**: éxito sin cuerpo de respuesta (ej. DELETE).
- **400 Bad Request**: error de validación/entrada del cliente.

- **401 Unauthorized:** falta autenticación o es inválida.
- **403 Forbidden:** autenticado pero sin permisos.
- **404 Not Found:** recurso no existe.
- **409 Conflict:** conflicto de estado (ej. recurso duplicado, optimistic locking).
- **422 Unprocessable Entity:** sintaxis correcta pero semánticamente inválida.
- **500 Internal Server Error:** error no controlado del servidor.

¿Cómo versionas una API?

- **En la URL:** /api/v1/usuarios — la más simple y explícita, ampliamente usada.
- **En el header:** Accept: application/vnd.miapi.v2+json — más “RESTful puro” pero menos visible/descubrible.
- **Por query param:** /usuarios?version=2 — menos común, útil en casos puntuales.

En la práctica, versionar por URL es lo más pragmático y fácil de documentar/monitorear, aunque versionar por header se considera más alineado con los principios REST.

¿Qué es idempotencia?

Una operación es idempotente si ejecutarla una o varias veces con los mismos parámetros produce el mismo resultado/estado final, sin efectos secundarios adicionales por repetirla.

- **Idempotentes:** GET, PUT, DELETE (eliminar algo ya eliminado sigue dejando el recurso ausente). - **No idempotente por naturaleza:** POST (cada llamada puede crear un nuevo recurso), aunque se puede hacer idempotente usando una **idempotency key** enviada por el cliente, algo crítico en pagos o mensajería para evitar duplicados ante reintentos de red.

¿Cómo asegurarías una API REST?

- **HTTPS/TLS** siempre, nunca HTTP plano.
- **Autenticación:** OAuth2/JWT para APIs, o API keys para integraciones simples.
- **Autorización granular:** roles/scopes por endpoint (@PreAuthorize en Spring Security).
- **Validación estricta de entrada** (evitar inyección SQL, XSS).
- **Rate limiting** para prevenir abuso/DoS.
- **CORS** configurado correctamente (no * en producción).
- **Headers de seguridad** (CSP, HSTS, X-Content-Type-Options).
- **No exponer stack traces** ni información sensible en errores.
- Auditoría/logging de accesos sensibles.

¿Qué es OpenAPI/Swagger?

OpenAPI es una especificación estándar (en YAML/JSON) para describir APIs REST: endpoints, parámetros, esquemas de request/response, códigos de error, seguridad, etc.

Swagger es el conjunto de herramientas (Swagger UI, Swagger Editor) que consumen esa especificación para generar documentación interactiva y permitir “probar” los endpoints desde el navegador. En Spring Boot se usa comúnmente springdoc-openapi (que

reemplazó a Springfox, migración que hiciste en hogar-hr) para generar la especificación automáticamente desde el código.

¿Cómo documentas tus servicios?

- Documentación **auto-generada** con OpenAPI/Swagger directamente desde anotaciones en el código (`@Operation`, `@Schema`), para que nunca quede desactualizada respecto al código real.
 - **README** por servicio con propósito, cómo levantarlo localmente, variables de entorno necesarias.
 - Diagramas de arquitectura/flujo (C4 model o similar) para contexto de alto nivel.
 - Ejemplos de request/response reales en la documentación para facilitar la integración de otros equipos.
 - Registro de decisiones de arquitectura (ADRs) para contexto histórico del “por qué” de ciertas decisiones.
-

5. Bases de Datos

Diferencia entre SQL y NoSQL.

- **SQL** (relacional: PostgreSQL, SQL Server, MySQL): esquema fijo, tablas relacionadas por claves foráneas, transacciones ACID fuertes, ideal para datos altamente estructurados y con relaciones complejas.
- **NoSQL** (MongoDB, Cassandra, Redis, etc.): esquema flexible/dinámico, optimizado para escalabilidad horizontal y alto volumen, modelo de consistencia normalmente eventual (aunque varía por producto). Tipos: documentales (MongoDB), clave-valor (Redis), columnares (Cassandra), de grafos (Neo4j).
- Elección depende del caso: relaciones complejas y consistencia fuerte → SQL; escalabilidad masiva, esquema cambiante, o datos semi-estructurados → NoSQL.

¿Qué es un índice y cuándo lo usarías?

Un índice es una estructura de datos adicional (normalmente un B-Tree) que acelera las búsquedas en una tabla, a costa de espacio extra y de ralentizar ligeramente las escrituras (inserts/updates), ya que el índice también debe actualizarse.

Úsalo cuando: - Una columna se usa frecuentemente en WHERE, JOIN o ORDER BY. - Hay alta cardinalidad (muchos valores distintos) — un índice sobre una columna booleana, por ejemplo, aporta poco. - Evita sobre-indexar tablas con muchas escrituras, ya que cada índice adicional penaliza el rendimiento de insert/update/delete.

Explica los niveles de aislamiento de transacciones.

De menor a mayor aislamiento (y menor a mayor costo de rendimiento): 1. **Read Uncommitted**: permite “dirty reads” (leer datos no confirmados de otra transacción). 2. **Read Committed** (default en PostgreSQL y SQL Server): solo lees datos confirmados, pero puede haber “non-repeatable reads” (releer la misma fila da valores distintos dentro de la misma transacción). 3. **Repeatable Read** (default en MySQL/InnoDB): la misma fila da el

mismo valor durante toda la transacción, pero puede haber “phantom reads” (nuevas filas aparecen en un rango consultado). 4. **Serializable**: máximo aislamiento, las transacciones se comportan como si se ejecutaran secuencialmente. Más seguro, pero mayor contención y menor throughput.

¿Qué es una consulta N+1?

Problema de rendimiento donde, para obtener una lista de N entidades más sus relaciones, se ejecuta 1 consulta para traer la lista y luego N consultas adicionales (una por cada entidad) para traer sus relaciones asociadas — en vez de una sola consulta optimizada con JOIN.

Ejemplo típico en JPA/Hibernate: traer una lista de Pedido y por cada uno acceder a `pedido.getCliente()` con fetch tipo LAZY, disparando una query por cada pedido.

Soluciones: usar JOIN FETCH en JPQL, @EntityGraph, o configurar el fetch adecuado según el caso de uso.

¿Cómo optimizarías una consulta lenta?

1. Analizar el plan de ejecución (EXPLAIN ANALYZE en PostgreSQL, EXPLAIN en MySQL/SQL Server) para ver si usa índices o hace full table scan.
2. Agregar índices en columnas usadas en WHERE/JOIN/ORDER BY si faltan.
3. Evitar SELECT *; traer solo las columnas necesarias.
4. Revisar si hay N+1 queries desde el ORM.
5. Considerar paginación (LIMIT/OFFSET o keyset pagination para grandes volúmenes).
6. Revisar si la consulta puede reescribirse (subqueries innecesarias, funciones sobre columnas indexadas que invalidan el índice).
7. Si es un patrón de lectura frecuente y los datos cambian poco, considerar caché (Redis) o una vista materializada.

Diferencia entre JOIN, LEFT JOIN e INNER JOIN.

- **INNER JOIN** (o simplemente JOIN): devuelve solo las filas donde hay coincidencia en ambas tablas.
- **LEFT JOIN** (LEFT OUTER JOIN): devuelve todas las filas de la tabla izquierda, y las coincidencias de la derecha si existen (si no, NULL). Útil cuando quieres “todos los X, tengan o no un Y relacionado” (ej. todos los clientes, tengan o no pedidos).
- **RIGHT JOIN**: análogo pero priorizando la tabla derecha (menos usado en la práctica, se suele reescribir como LEFT JOIN invirtiendo el orden de tablas).

¿Qué experiencia tienes con PostgreSQL, SQL Server o MongoDB?

(Esta pregunta es personal — te dejo un borrador de respuesta basado en lo que hemos trabajado; ajústalo a tu experiencia real): tu background más fuerte está en **SQL Server con T-SQL** (procedimientos almacenados, vistas, funciones) en el sector financiero, además de trabajo reciente con **MariaDB/MySQL** (base de datos HRLOG con stored procedures para logging estructurado en el proyecto hogar-hr). Si te preguntan por PostgreSQL o MongoDB

específicamente y tu experiencia es limitada, es mejor ser honesto y mencionar tu solidez transferible en SQL relacional (T-SQL/PL-SQL son muy similares conceptualmente a PL/pgSQL), y mostrar disposición a adaptarte rápido.

6. Kafka / Mensajería

¿Qué es Apache Kafka?

Plataforma distribuida de streaming de eventos, diseñada para manejar grandes volúmenes de datos en tiempo real con alta durabilidad y throughput. Funciona como un log distribuido, particionado y replicado, donde los productores escriben eventos y los consumidores los leen de forma independiente (a diferencia de una cola tradicional, los mensajes no se eliminan al consumirse, sino que se retienen según una política de retención).

Diferencia entre Queue y Topic.

- **Queue** (cola, ej. RabbitMQ clásico): un mensaje es consumido por **un solo consumidor**; una vez procesado, se elimina (patrón punto a punto).
- **Topic** (Kafka, pub/sub): un mensaje puede ser leído por **múltiples consumidores/grupos independientes**, y el mensaje permanece en el log según la política de retención, no según si fue “consumido” — el consumo no elimina el mensaje, solo avanza el offset del consumidor.

¿Qué es un Consumer Group?

Conjunto de consumidores que cooperan para leer un topic, donde Kafka reparte las particiones del topic entre los miembros del grupo, de modo que cada partición es leída por **un solo consumidor dentro del grupo** (permitiendo paralelismo horizontal), mientras que grupos distintos pueden leer el mismo topic de forma completamente independiente entre sí.

¿Cómo garantizarías la entrega de mensajes?

- **At-least-once**: configurar `acks=all` en el productor (espera confirmación de todas las réplicas), y que el consumidor confirme (`commit`) el offset **después** de procesar exitosamente el mensaje, no antes.
- Usar **réplicas** (`replication.factor > 1`) para tolerar caída de brokers.
- Manejar reintentos con backoff en caso de fallos transitorios.
- Para garantías más fuertes, Kafka soporta **exactly-once semantics** (transacciones de productor + `isolation.level=read_committed` en el consumidor), aunque con mayor complejidad y costo de rendimiento.

¿Qué estrategias utilizarías para evitar mensajes duplicados?

- **Idempotencia del productor** (`enable.idempotence=true` en Kafka, evita duplicados por reintentos de red del propio productor).

- **Idempotencia del lado del consumidor:** diseñar el procesamiento para que aplicar el mismo mensaje dos veces no cause efectos duplicados (ej. usar un messageId único y verificar si ya fue procesado antes de aplicar el efecto, guardándolo en una tabla de deduplicación).
- **Transacciones de Kafka** para escenarios que requieren exactly-once end-to-end.
- Diseñar operaciones de negocio como idempotentes por naturaleza cuando sea posible (ej. “establecer estado = X” en vez de “incrementar contador”).

¿Qué experiencia tienes con RabbitMQ o IBM MQ?

(Personal — completar según tu experiencia real). Si no tienes experiencia directa, puedes responder honestamente destacando tu comprensión conceptual de mensajería (colas vs. topics, patrones pub/sub, garantías de entrega) y tu disposición para adaptarte rápidamente dado tu background en sistemas backend robustos.

7. Docker y Kubernetes

¿Qué es Docker?

Plataforma de contenedorización que empaqueta una aplicación junto con todas sus dependencias (librerías, runtime, configuración) en una unidad estándar y portable llamada **contenedor**, garantizando que funcione igual en cualquier entorno (“funciona en mi máquina” deja de ser un problema).

Diferencia entre imagen y contenedor.

- **Imagen:** plantilla inmutable de solo lectura que define el sistema de archivos y configuración de una aplicación (construida a partir de un Dockerfile, en capas).
- **Contenedor:** instancia en ejecución de una imagen, con una capa de escritura adicional encima. Puedes crear múltiples contenedores a partir de la misma imagen, cada uno con su propio estado en tiempo de ejecución, aislado de los demás.

¿Cómo optimizarías una imagen Docker?

- Usar imágenes base **ligeras** (alpine, distroless) en vez de imágenes completas de SO.
- **Multi-stage builds:** compilar en una etapa con todas las herramientas de build, y copiar solo el artefacto final a una imagen liviana de runtime.
- Minimizar el número de capas combinando comandos RUN relacionados.
- Aprovechar la **caché de capas** ordenando el Dockerfile de lo que cambia menos a lo que cambia más (ej. copiar pom.xml/package.json e instalar dependencias antes de copiar el código fuente).
- Usar .dockerignore para no copiar archivos innecesarios (logs, .git, node_modules).
- No incluir herramientas de debug/build en la imagen final de producción.

¿Qué es Kubernetes?

Plataforma de orquestación de contenedores de código abierto que automatiza el despliegue, escalado, balanceo de carga y gestión del ciclo de vida de aplicaciones contenerizadas a través de un clúster de máquinas, manteniendo el estado deseado definido declarativamente (ej. “quiero 3 réplicas de este servicio corriendo siempre”).

¿Qué es un Pod?

La unidad mínima de despliegue en Kubernetes. Representa uno o más contenedores que comparten red (misma IP) y almacenamiento, y que se despliegan siempre juntos en el mismo nodo. Normalmente un Pod contiene un solo contenedor principal, aunque puede incluir contenedores auxiliares (“sidecars”) para tareas como logging o proxy.

¿Qué es un Deployment?

Recurso de Kubernetes que gestiona un conjunto de Pods idénticos (vía ReplicaSets) declarando el **estado deseado**: cuántas réplicas deben existir, qué imagen usar, estrategia de actualización, etc. Kubernetes se encarga de mantener ese estado automáticamente (si un Pod muere, lo recrea; si escalas de 3 a 5 réplicas, crea las que faltan).

¿Qué es un ConfigMap?

Objeto de Kubernetes que permite desacoplar la configuración (variables de entorno, archivos de configuración) de la imagen del contenedor, para poder cambiar configuración sin reconstruir la imagen. Se puede inyectar como variables de entorno o como archivos montados en el Pod. Para datos sensibles (contraseñas, tokens) se usa el equivalente **Secret**, que además se almacena codificado/cifrado.

¿Cómo realizarías un rollout y rollback?

- **Rollout**: al actualizar la imagen en un Deployment (`kubectl set image deployment/mi-app mi-app=imagen:v2`), Kubernetes ejecuta la estrategia configurada — típicamente **RollingUpdate**, reemplazando Pods viejos por nuevos gradualmente, verificando `readinessProbe` antes de enviar tráfico, sin downtime.
- Monitorear el progreso con `kubectl rollout status deployment/mi-app`.
- **Rollback**: si algo falla, `kubectl rollout undo deployment/mi-app` regresa automáticamente a la revisión anterior (Kubernetes mantiene un historial de revisiones del Deployment).
- Buenas prácticas: configurar `readinessProbe`/`livenessProbe` adecuados, y `maxUnavailable`/`maxSurge` para controlar qué tan agresivo es el despliegue.

Consejo general para la entrevista: dado tu perfil (fuerte en .NET/C# con experiencia reciente en Java/Spring Boot vía el proyecto hogar-hr), es buena idea conectar tus respuestas con ejemplos concretos de tu propia experiencia cuando puedas — por ejemplo, mencionar la migración de Springfox a springdoc-openapi, el `appenders` custom de Log4j2, o el uso de `Mono<Void>` con Reactor para logging, muestra experiencia real y no solo teoría memorizada.